BE-05: Backend Component Diagram

Intelligent Kanban System with an AI Assistant

Symbat Saparbay | Akbope Bakytkeldy

YouTrack: DEMO-24

1 Purpose

This document opens the “Intelligent Kanban System” box from the context diagram (BE-04, DEMO-23). It shows:

- the containers that make up the system;
- the components inside the backend API and the AI worker (C4 level 3);
- how the components work together in the key scenarios;
- the life cycle of an AI command and how the components map to code modules.

2 Container Diagram

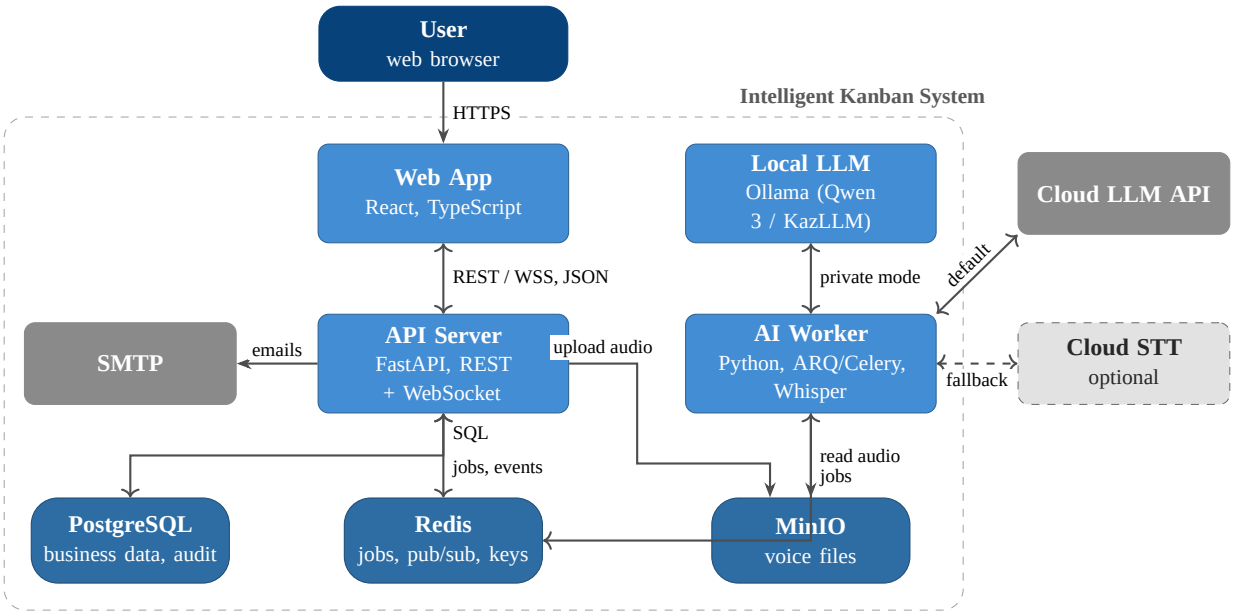


Figure 1: Containers of the system. Only the API Server writes business data; the AI Worker produces transcripts and action plans.

Container	Technology	Responsibility
Web App	React, TypeScript, Vite, dnd-kit, TanStack Query	Kanban UI, voice recording, preview and confirmation of AI actions, real-time updates

Container	Technology	Responsibility
API Server	Python 3.12, FastAPI, SQLAlchemy 2.0	Authentication, access control, all business logic and data changes, AI command validation and execution, WebSocket events
AI Worker	Python, ARQ/Celery, faster-whisper, transformers	Speech-to-text with language routing, command parsing with an LLM, periodic jobs (reminders, email, retention clean-up)
Local LLM	Ollama with Qwen 3 or KazLLM 8B	Command parsing in private mode and as a fallback
PostgreSQL	PostgreSQL 16, pg_trgm	Users, workspaces, projects, boards, tasks, comments, history, AI commands, audit log
Redis	Redis 7	Job queue, pub/sub for WebSocket fan-out, idempotency keys, rate limits
MinIO	S3-compatible storage	Encrypted voice files with a retention period

3 Backend Components (C4 Level 3)

The API Server follows a layered structure: routers handle HTTP and WebSocket, services contain business rules, repositories and clients access storage. The AI Worker has its own components but writes results only through the AI command records, never directly into tasks.

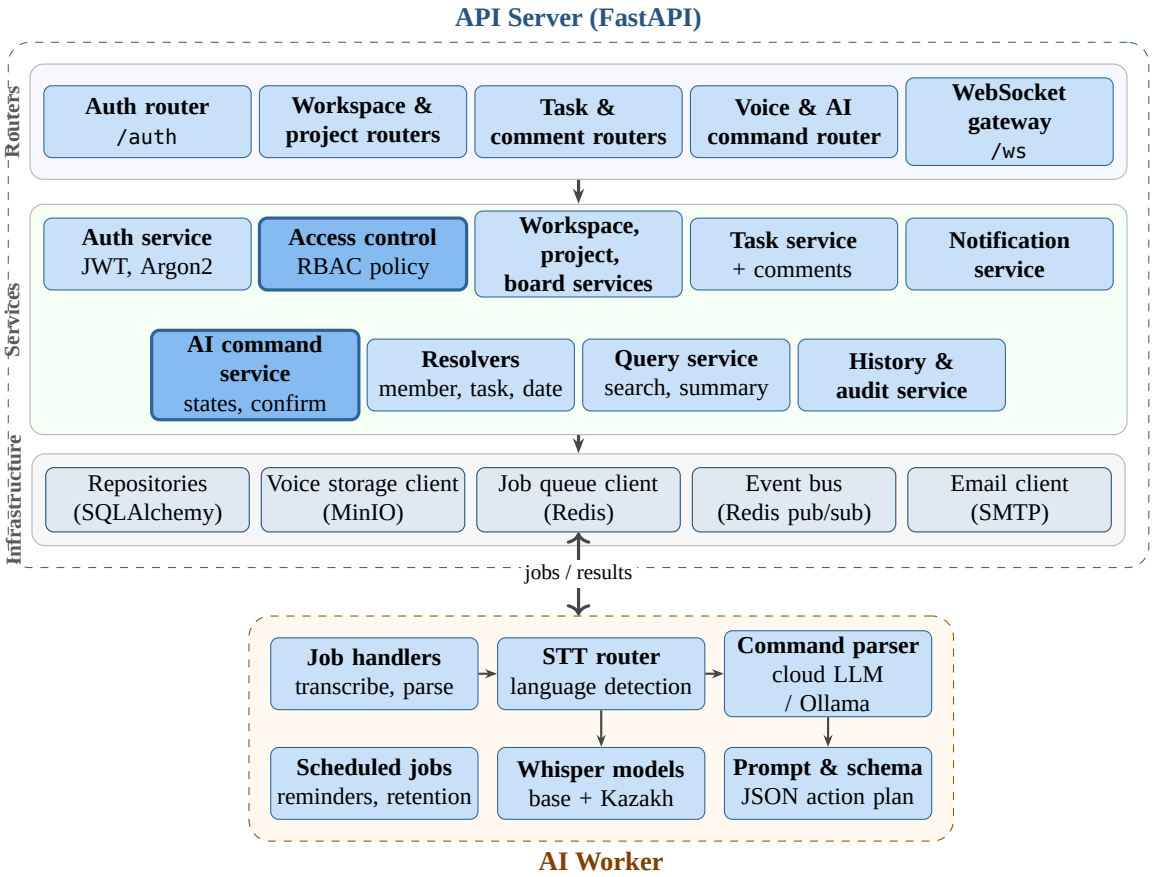


Figure 2: Components of the API Server and the AI Worker. Thick boxes are the components that enforce the “AI only proposes” rule.

3.1 Component Responsibilities

Component	Responsibility	Depends on
API Server — routers		
Auth router	Register, log in, refresh and revoke tokens	Auth service
Workspace & project routers	CRUD for workspaces, projects, boards, columns; membership and roles	Access control, workspace/project/board services
Task & comment routers	CRUD for tasks, moving between columns, comments, task history	Access control, task service, history
Voice & AI command router	Upload audio or text command, edit transcript, view proposed plan, confirm or reject, poll status	AI command service
WebSocket gateway	Authenticated connection per user; pushes board changes, AI command status and notifications	Event bus, access control
API Server — services		
Auth service	Password hashing (Argon2), JWT issue and validation, refresh-token rotation	User repository
Access control	One place for permission rules: workspace roles (owner, admin, member) and project roles (manager, member, viewer); used by every service, including AI actions	Membership repository
Workspace / project / board services	Business rules for workspaces, projects, boards and columns, member invitations	Repositories, access control
Task service	Create, update, delete, assign, change status, deadline and priority; column order; comments. The **same methods** serve manual UI actions and confirmed AI actions	Repositories, access control, history, notifications, event bus
Notification service	Builds in-app and email notifications from domain events; respects user preferences	Event bus, email job
AI command service	Creates AI commands, stores transcripts and plans, runs the state machine (Section 4), validates plans with Pydantic, executes confirmed actions through the task service, guarantees idempotent confirmation	Resolvers, task service, query service, job queue
Resolvers	Turn mentions into IDs: **member resolver** (Kazakh case endings, transliteration, fuzzy match), **task resolver** (pg_trgm over accessible tasks), **date resolver** (relative dates in the user's time zone). Never invent values: no match or several matches become a clarification	Repositories
Query service	Executes Search Tasks and Project Summary as database queries built from the LLM filter; returns only accessible records	Repositories, access control
History & audit service	Writes `task_history` (field, old value, new value, author, source UI/AI) and `audit_log`	Repositories
API Server — infrastructure		
Repositories	SQLAlchemy data access, transactions	PostgreSQL
Voice storage client	Upload, presigned URLs, deletion	MinIO
Job queue client	Enqueue jobs, receive results	Redis
Event bus	Publish domain events, fan-out to WebSocket gateways on all instances	Redis pub/sub
Email client	Send emails with retries	SMTP
AI Worker		
Job handlers	`transcribe(command_id)`, `parse(command_id)`; update the AI command record with results or errors	STT router, command parser

Component	Responsibility	Depends on
STT router	Language detection restricted to kk/ru/en; routing to the Kazakh or base model (kk top or $p(\text{kk}) \geq 0.30$); user hint overrides detection	Whisper models, cloud STT (fallback)
Command parser	Single `CommandParser` interface with two providers (cloud LLM, Ollama); returns a JSON action plan	Prompt & schema, LLM providers
Prompt & schema	System prompt, list of the eight actions, JSON Schema of the plan, context (member names, today's date, time zone)	—
Scheduled jobs	Deadline reminders every 15 minutes, email sending, deletion of voice files after the retention period	Repositories, voice storage, email client

4 AI Command Life Cycle

Each voice or text command is stored as an `ai_commands` record. Its status changes only through the AI command service.

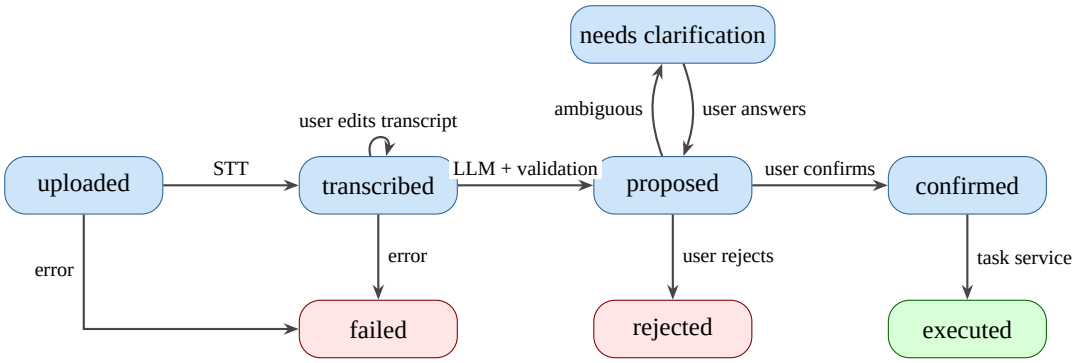


Figure 3: States of an AI command. Data changes happen only on the transition *confirmed* → *executed*.

- **Idempotency.** `POST /ai-commands/{id}/confirm` moves *proposed* → *confirmed* in one database transaction with a row lock. A repeated request on a *confirmed* or *executed* command returns the existing result and creates nothing.
- **Partial confirmation.** For a plan with several tasks (meeting summary, several assignments), the user can accept, edit or reject each action; only accepted actions are executed.
- **Errors.** STT or LLM failures move the command to *failed* with a readable reason; the user can retry or type the command manually.
- **Expiry.** Proposed commands that are not confirmed within 24 hours are marked as rejected, so an old plan cannot be executed on changed data.

5 Key Scenarios

5.1 Create a Task by Voice (Case 1)

#	Component	Action
1	Web App	Records audio and sends it with the current project ID to `POST /voice-commands`
2	Voice & AI command router	Checks that the user is a member of the project (access control)

#	Component	Action
3	AI command service	Saves audio through the voice storage client, creates an AI command (*uploaded*), enqueues `transcribe`
4	AI Worker: STT router	Detects the language, routes to the Kazakh or base model, stores the transcript (*transcribed*)
5	Web App	Shows the transcript; the user can correct it
6	AI Worker: command parser	Sends the transcript, project member names, date and time zone to the LLM; receives the JSON plan
7	AI command service	Validates the plan with Pydantic; resolvers map “Данияр” to a project member and “до пятницы” to a date; unresolved fields are flagged (*proposed* or *needs clarification*)
8	Web App	Shows the preview card via WebSocket; the user edits fields and confirms
9	AI command service	Idempotent confirm, then calls the task service
10	Task service	Creates the task with permission checks, writes history and audit, publishes `task.created`
11	WebSocket gateway, notification service	Board updates for all members; the assignee receives an in-app and email notification

5.2 Update or Change Status of an Existing Task (Cases 3, 4)

The command parser returns an `update_task` or `change_status` action with a *description* of the task. The task resolver searches accessible tasks (pg_trgm on titles): one match goes to the preview, several matches produce a list to choose from, no match produces “task not found”. After confirmation the same task service methods as in the UI are used.

5.3 Search Tasks and Project Summary (Case 5)

The LLM returns only a filter, for example `{status: not done, deadline_before: today}`. The query service runs the query on records accessible to the user and returns real rows. For a summary, only these rows are passed to the LLM for wording, and every task in the answer links to a real task ID. These actions do not change data and need no confirmation.

5.4 Deadline Reminders

A scheduled job in the AI Worker runs every 15 minutes, selects tasks due within 24 hours or overdue that have not been notified yet, and publishes events; the notification service creates in-app notifications and enqueues emails.

6 Code Structure

Components map directly to Python packages of the backend repository:

```
backend/
  app/
    main.py                # FastAPI app, routers, middleware
    core/                  # config, security (JWT, Argon2), errors
    api/                   # routers: auth, workspaces, projects,
                           #   boards, tasks, comments, ai_commands, ws
    services/              # auth, access_control, workspaces, projects,
                           #   boards, tasks, notifications, history_audit,
```

```
                          #   query, ai_commands
    ai/
      resolvers/          # member, task, date
      schemas.py          # Pydantic models of the 8 actions and the plan
    repositories/         # SQLAlchemy repositories
    models/               # SQLAlchemy models
    infra/                # minio, redis queue, event bus, email
  worker/
    jobs/                 # transcribe, parse, reminders, retention, email
    stt/                  # STT router, Whisper wrappers
    llm/                  # CommandParser, cloud and Ollama providers, prompt
  migrations/             # Alembic
  tests/                  # unit, API, AI scenarios
```

The `ai/schemas.py` module is shared by the API Server and the AI Worker, so the plan format is defined once.

7 Cross-Cutting Rules

Rule	Implementation
AI only proposes	The AI Worker has no access to task tables; it writes only transcripts and plans to `ai_commands`. Data changes go through the task service after confirmation
One permission model	Every router and service calls access control; AI actions run with the permissions of the user who confirmed them
Transactions	Each confirmed action (task change, history, audit, outbox event) is saved in one transaction
Error handling	Typed domain errors mapped to HTTP codes (400 validation, 403 forbidden, 404 not found, 409 conflict, 503 AI unavailable) with a readable message
Configuration	LLM provider, STT fallback, Kazakh threshold, retention period and reminder interval are environment settings, not code
Observability	Structured logs with command ID; timings of STT and LLM steps are stored with each AI command for the benchmark and the test cases